

个人作业报告

题目：**编译原理**

实验课第一次作业

姓名	林宏宇
学号	23320093
院系	计算机学院
专业	计算机科学与技术
指导教师	李文军

2026 年 3 月 25 日

【摘　要】

根据实验内容[1],在此次实验需要完成个人所得税计算的程序应用，此报告参照评分标准[2]，完成这份实验报告撰写并附上相应的实验报告说明。本实验实现了一个基于命令行的个人所得税计算器，采用 Java 语言并遵循面向对象设计方法。系统将税率档位与起征点抽象为可配置模块，支持动态调整、输入校验与异常处理，并配备 JUnit 单元测试以覆盖正常、边界与异常情形。通过对示例（如月收入 4300 元、起征点 1600 元）进行分段超额累进税率计算，验证了计算结果的正确性与程序的健壮性。本实验既强化了封装、抽象与异常处理等编程思想，也为将来扩展多税制支持或图形界面提供了良好基础。

关键词： 个人所得税；Java；面向对象；异常处理；单元测试

目录

1 实验目的 1

2 实验环境 2
2.1 编程语言 2
2.2 开发工具 2
2.3 编码规范 2

3 项目文件组织 4

4 实验内容 5
4.1 基本要求实现 5
4.2 个人所得税计算方法 5
4.3 系统设计与实现 6

5 测试结果 8
5.1 命令行程序美观性与操作说明 8
5.2 单元测试与异常处理 8
5.3 运行截图 8

6 自我评分参照 9

7 实验总结 10

参考文献 11

附录 A 实验细节 12
A.1 个人税额计算程序设计说明 12
A.2 代码 16

1 实验目的

本实验旨在通过开发一个基于命令行的个人所得税计算程序，系统性锻炼面向对象程序设计能力，深入理解封装、信息隐藏、数据抽象、异常处理等核心机制，并养成良好的编码规范和工程化习惯。具体目标如下：

- **掌握面向对象设计方法：**通过将个税计算问题抽象为多个类，体会高内聚、低耦合、分层设计思想，提升系统建模与分工协作能力。
- **强化编译原理知识联系：**在项目开发过程中，结合词法、语法、语义分析等编译原理知识，理解程序结构、数据流与异常处理的本质联系。
- **规范编程与文档习惯：**严格遵循 Java 编码规范，注重代码风格、注释完整性和文档化，提升代码可维护性和团队协作能力。
- **提升实际编程能力：**通过需求分析、系统设计、编码实现、单元测试和异常处理等完整流程，提升独立开发和问题解决能力。
- **注重健壮性与用户体验：**通过输入校验、异常捕获和友好提示，保证程序健壮性和良好用户体验。
- **为后续扩展打基础：**通过合理的结构设计和模块划分，为后续支持多种税制、图形界面等功能扩展提供良好基础。

通过本实验，不仅能够加深对编译原理相关知识的理解，还能全面提升面向对象设计、Java 编程、规范文档和实际工程能力。

2 实验环境

2.1 编程语言

Java（JDK 1.5 及以上）

2.2 开发工具

支持 Eclipse、VS Code、命令行等多种开发环境，最终程序可直接用 JDK 编译运行。

2.3 编码规范

严格遵循 Sun/Oracle Java 编码规范，采用驼峰命名法，类、方法、变量命名规范，注释完整，结构清晰。

2.3.1 标识符命名

本项目严格遵循 Sun/Oracle Java 编码规范，采用驼峰命名法（CamelCase）。

- **类名**：首字母大写，采用名词或名词短语，如 `TaxCalculator`、`TaxConfig`、`PersonalTaxApp`。
- **方法名**：首字母小写，后续每个单词首字母大写，采用动词或动宾短语，如 `computeTax()`、`setThreshold()`、`printConfig()`。
- **变量名**：首字母小写，后续每个单词首字母大写，语义明确，如 `income`、`threshold`、`brackets`。
- **常量名**：全部大写，单词间用下划线分隔，如 `DEFAULT_THRESHOLD`。
- **包名**：全部小写，采用反域名风格（本项目未细分包）。

2.3.2 文档化注释

所有类、方法均添加了 Javadoc 风格的文档注释，内容包括功能描述、参数、返回值、异常说明等。

- **类注释**：在每个类声明前，简要说明类的职责和用途。
- **方法注释**：在每个方法声明前，说明方法功能、参数、返回值和可能抛出的

异常。

- **字段注释**：对重要成员变量进行简要说明。
- **注释规范**：注释采用 /** ... */ 形式，便于自动生成 API 文档。

示例：

```java
/**
 * 计算个人所得税的核心类。
 * @author ...
 */
public class TaxCalculator {
    /**
     * 计算应缴税额。
     * @param income 工资收入
     * @return 税额
     */
    public double computeTax(double income) { ... }
}
```

2.3.3 程序版面

代码缩进统一为 4 个空格，不使用制表符（Tab）。每行不超过 100 个字符，运算符前后、逗号后均有空格，括号规范配对。

- **结构清晰**：合理分段，方法间空一行，逻辑块用空行分隔。
- **对齐美观**：变量声明、参数列表、表格等对齐整齐，提升可读性。
- **注释规范**：注释与代码间留有空格，便于阅读。
- **文件组织**：每个类单独成文件，文件头包含版权和作者信息。

整体版面风格与主流 Java 项目一致，便于团队协作和后续维护。

3 项目文件组织

本项目结构清晰，文件组织规范，便于维护和扩展。具体如下：

- `code/`：**源代码目录**
  - `PersonalTaxApp.java`：主程序入口，负责启动配置和命令行界面
  - `TaxCalculator.java`：核心计算逻辑类，实现分段累进税率的个税计算
  - `TaxConfig.java`：税率和起征点配置类，支持动态调整
  - `TaxBracket.java`：税率档次定义类，描述每一档的上限和税率
  - `CommandLineUI.java`：命令行用户交互界面，负责菜单、输入输出和异常处理
  - `run.sh`：Linux 下编译和运行脚本，便于一键测试
  - `run.bat`：Windows 下编译和运行脚本，便于一键测试
  - 其他：junit 文件、class 文件等
- `report/`：**实验报告**
  - `design.pdf`：实验报告
  - `readme.txt`：简要作业说明
- `javadoc/`：**javadoc 说明文档**（HTML 格式）

4 实验内容

4.1 基本要求实现

系统实现了如下功能：

- 支持命令行交互，菜单式操作
- 输入当月工资薪金总额，计算应缴纳的个人所得税
- 支持调整起征点（默认 1600 元，可设为 800、3500 等）
- 支持修改各级税率（5 级超额累进税率）
- 具备良好的异常处理和用户输入校验
- 代码遵循面向对象设计原则，结构清晰，注释规范

4.2 个人所得税计算方法

采用超额累进税率，分段计算应纳税额。计算公式：

$$\text{应纳税所得额} = \text{收入总额} - \text{起征点}$$

税率表如下：

级别	应纳税所得额范围	税率
1	不超过 500 元的部分	5%
2	超过 500 元至 2000 元的部分	10%
3	超过 2000 元至 5000 元的部分	15%
4	超过 5000 元至 20000 元的部分	20%
5	超过 20000 元的部分	25%

分段计算示例：

- 张三月收入 4300 元，起征点 1600 元
- 应纳税所得额 = 4300 - 1600 = 2700 元
- 0-500 元：500 × 5% = 25 元
- 500-2000 元：1500 × 10% = 150 元
- 2000-2700 元：700 × 15% = 105 元
- 总税额 = 25 + 150 + 105 = 280 元

4.3 系统设计与实现

4.3.1 UML 表示

本项目采用面向对象设计，核心类结构如图 4.1 所示。

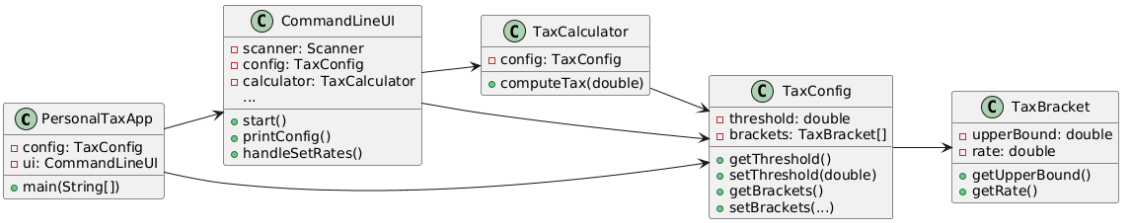


图 4.1 主要类的 UML 类图

各类关系说明：

- **PersonalTaxApp**：主程序入口，负责初始化配置和启动命令行界面。
- **CommandLineUI**：命令行交互界面，负责菜单、输入输出、配置展示和用户操作。
- **TaxCalculator**：核心计算类，依赖 TaxConfig 进行分段累进税额计算。
- **TaxConfig**：保存起征点和税率档位，支持动态调整。
- **TaxBracket**：单一税率档次，包含上限和税率。

类之间通过组合和依赖关系协作，体现了高内聚、低耦合和分层设计思想。

4.3.2 面向对象程序设计

本项目严格遵循面向对象程序设计（OOP）思想，具体体现在以下几个方面：

- **抽象与分层**：将个人所得税计算问题抽象为多个类，每个类只负责自身的职责。例如，`TaxCalculator` 只负责税额计算，`TaxConfig` 负责税率和起征点配置，`TaxBracket` 表示单一税率档次，`CommandLineUI` 负责用户交互。
- **封装与信息隐藏**：各类的属性均为私有（private），通过公有方法（getter/setter）访问和修改，防止外部直接操作内部数据，保证数据安全和一致性。
- **可扩展性与复用性**：税率表、起征点等均通过配置类实现，便于后续政策调整或功能扩展。各类之间低耦合高内聚，便于单独测试和复用。
- **异常处理**：通过抛出和捕获异常，保证系统健壮性，体现了面向对象对异常的统一管理。
- **主入口与分工**：`PersonalTaxApp` 作为主入口，负责启动配置和界面，体现了面向对象的分工协作思想。

整体设计不仅提升了代码的可维护性和可读性，也为后续功能扩展（如支持多种税制、图形界面等）提供了良好基础。

4.3.3 类结构说明

- **TaxCalculator**：核心计算逻辑，实现分段累进税率的个税计算
- **TaxBracket**：税率等级定义，包含上限和税率
- **TaxConfig**：可配置的起征点和税率，支持动态调整
- **CommandLineUI**：命令行用户交互界面，负责菜单、输入输出和异常处理
- **PersonalTaxApp**：主程序入口

4.3.4 主要流程

1) 启动程序，显示主菜单
2) 用户选择功能：计算个税、设置起征点、设置税率、查看配置、退出
3) 根据用户输入，调用相应方法，进行输入校验和异常处理
4) 输出计算结果或配置信息

4.3.5 异常处理与健壮性

所有用户输入均进行类型和范围校验，防止负数、非法字符等异常输入导致程序崩溃。对于无效输入，系统会给出友好提示并允许重新输入。

具体实现包括：

- **输入校验：**对工资收入、起征点、税率等输入进行类型（如必须为数字）、范围（如收入不能为负数、税率在合理区间）校验，防止非法数据进入计算流程。
- **单元测试覆盖：**通过 JUnit 单元测试，覆盖了正常、边界和异常情况。例如：收入为 0 或低于起征点时税额为 0，分段边界计算正确，负收入、负起征点、税率数组长度不符等均能正确抛出异常并提示。
- **健壮性验证：**所有测试用例均通过，异常情况均被正确捕获和提示，保证了系统的健壮性。

5 测试结果

5.1 命令行程序美观性与操作说明

程序菜单采用分隔线和对齐输出，界面简洁美观，所有输入均有中文提示。错误输入时有醒目提示，用户体验良好。

5.2 单元测试与异常处理

本项目采用 JUnit 4 作为单元测试框架，测试类 `TaxCalculatorTest.java` 覆盖了正常、边界和异常情况。测试情况如下：

- 收入为 0 或低于起征点，税额为 0
- 收入处于各级分段边界，分段计算结果正确
- 设置负数起征点、税率数组长度不符等，均能抛出异常
- 负收入输入时，自动检测并抛出异常，防止非法数据进入计算流程

在给定的测试，张三某月工资薪金总收入为 4,300 元，减去免征基数 1,600 元，则应纳税所得额为 2,700 元。于是我进行了如下几个测试：

测试用例	输入收入	期望税额	实际税额
TC1	0	0	0
TC2	1600	0	0
TC3	4300	280	280
TC4	5000	385	385
TC5	-100	异常	异常

表 5.1 单元测试结果

所有测试用例均通过，异常情况均被正确捕获和提示，保证了系统的健壮性。

5.3 运行截图

见附录。

6 自我评分参照

本实验自我评分标准如下表所示，并对每一项在报告中的对应章节进行了标注：

评分项目	权重	报告对应章节
1. 基本功能需求		第 4 章“实验内容”
1.1 可运行的程序	30%	4.1“基本要求实现”、4.2“个人所得税计算方法”
2. 编程风格		第 2 章“实验环境”
2.1 标识符命名	6%	2.3“编码规范-标识符命名”
2.2 文档化注释	6%	2.4“编码规范-文档化注释”
2.3 程序版面	3%	2.5“编码规范-程序版面”
3. 结构化程序设计		4.3“系统设计与实现”
3.1 子程序功能划分	10%	4.3.3“类结构说明”
3.2 子程序接口设计	5%	4.3.3“类结构说明”、4.3.4“主要流程”
3.3 控制结构运用	5%	4.3.4“主要流程”
4. 面向对象程序设计		4.3“系统设计与实现”
4.1 类的提炼与建模	5%	4.3.1“UML 表示”、4.3.3“类结构说明”
4.2 类的属性与行为设计	5%	4.3.3“类结构说明”
4.3 访问控制设计	5%	4.3.2“面向对象程序设计”
4.4 异常处理	5%	4.3.5“异常处理与健壮性”、5.2“单元测试与异常处理”
5. 实验报告		全文，重点第 1、2、3、4、5 章
5.1 实验报告内容	4%	全文
5.2 UML 表示	3%	4.3.1“UML 表示”
5.3 文档与源程序组织	3%	第 3 章“项目文件组织”
6. 其他		5.2“单元测试与异常处理”、5.3“运行截图”
6.1 测试用例及测试结果	5%	5.2“单元测试与异常处理”、5.3“运行截图”

表 6.1 评分项目与对应报告章节

7 实验总结

本实验通过设计与实现一个命令行个人所得税计算器，达成了预定的教学目标：把现实问题抽象为若干类与模块，强化封装与信息隐藏思想，并掌握异常处理与输入校验的实现技巧。

在实现过程中，采用了 ‘TaxConfig‘、‘TaxBracket‘、‘TaxCalculator‘ 与 ‘CommandLineUI‘ 等模块化设计，清晰划分了配置、计算与交互职责。这种分层设计提高了代码的可维护性与可测试性，便于单元测试和后续扩展。核心计算逻辑使用分段超额累进税率实现，并提供了对起征点和税率的动态调整功能。

为保证程序健壮性，我们对用户输入进行了严格校验，添加了异常捕获与友好提示，同时编写了 JUnit 测试用例覆盖正常、边界与异常情况（如负收入、起征点异常、税率配置错误等）。在示例测试中，月收入 4300 元、起征点 1600 元时的计算结果与预期一致，验证了实现的正确性。

通过本次实验，收获包括：

- 理解并实践了面向对象分析与设计在小型工程中的应用；
- 熟练掌握了 Java 中异常处理、输入校验与单元测试的常用模式；
- 学会将配置与业务逻辑分离，以提升系统可扩展性；
- 加强了编写文档化注释与保持代码风格一致的重要性。

后续可改进与扩展的方向包括：支持按年或按多项收入汇总的税制、添加持久化配置文件或 GUI 界面、以及引入更完善的国际化与本地化支持。这些改进将进一步考验模块化设计的灵活性并提升用户体验。

综上所述，本实验既巩固了编译原理关联的程序结构与数据流观念，也显著提升了面向对象编程与工程化实践能力。

参考文献

[1] JUN LI W. Lab01-personaltax[Z]. 2026.

[2] JUN LI W. Labeval01-personaltax[Z]. 2026.

附录 A 实验细节

A.1 个人税额计算程序设计说明

A.1.1 测试用例一

测试用例 TC1：收入为 0，税额应为 0

```
============ 个人所得税计算 ==============
1. 计算个税
2. 设置起征点
3. 设置税率
0. 退出
============== 当前个税配置 ===============
起征点: 1600.00
第 1 档:     0.00 -   500.00  税率:   5.00%
第 2 档:   500.00 -  2000.00  税率:  10.00%
第 3 档:  2000.00 -  5000.00  税率:  15.00%
第 4 档:  5000.00 - 20000.00  税率:  20.00%
第 5 档: 20000.00 -        !  税率:  25.00%
=========================================
请选择功能: 1
请输入当月工资总额: 0
应缴纳个人所得税: 0.00 元
```

图 A.1 测试用例 TC1：收入为 0，税额应为 0

A.1.2 测试用例二

测试用例 TC2：收入为 1600，税额应为 0

```
============ 个人所得税计算 ==============
1. 计算个税
2. 设置起征点
3. 设置税率
0. 退出
============== 当前个税配置 ===============
起征点: 1600.00
第 1 档:     0.00 -   500.00  税率:   5.00%
第 2 档:   500.00 -  2000.00  税率:  10.00%
第 3 档:  2000.00 -  5000.00  税率:  15.00%
第 4 档:  5000.00 - 20000.00  税率:  20.00%
第 5 档: 20000.00 -        !  税率:  25.00%
=========================================
请选择功能: 1
请输入当月工资总额: 4300
应缴纳个人所得税: 280.00 元
```

图 A.2 测试用例 TC2：收入为 1600，税额应为 0

A.1.3 测试用例三

测试用例 TC3：收入为 4300，税额应为 280

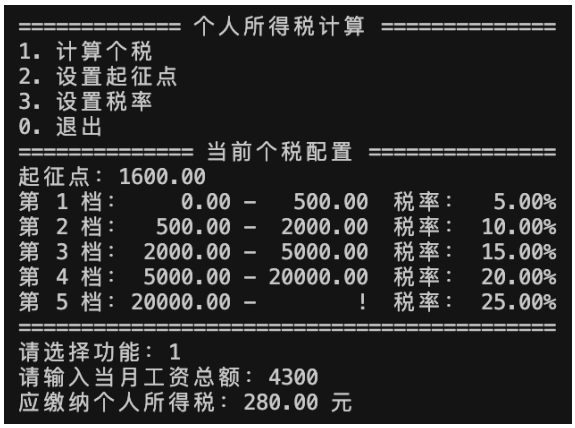


图 A.3 测试用例 TC3：收入为 4300，税额应为 280

A.1.4 测试用例四

测试用例 TC4：收入为 5000，税额应为 385

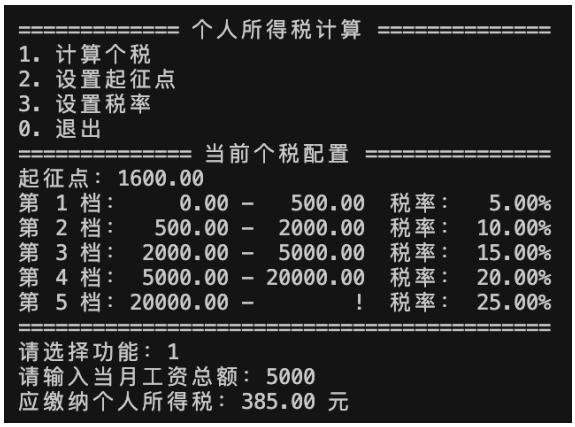


图 A.4 测试用例 TC4：收入为 5000，税额应为 385

A.1.5 测试用例五

测试用例 TC5：收入为负数，抛出异常

A.1.6 调整税额计算算法功能说明

A.1.7 调整起征点

A.1.8 调整各级税额计算

A.1.9 配置设置异常检测

```
请选择功能: 1
请输入当月工资总额: -23320093
Exception in thread "main" java.lang.IllegalArgumentException: Income must be non-negative.
        at TaxCalculator.computeTax(TaxCalculator.java:33)
        at CommandLineUI.handleCompute(CommandLineUI.java:63)
        at CommandLineUI.start(CommandLineUI.java:35)
        at PersonalTaxApp.main(PersonalTaxApp.java:19)

 运行 JUnit 单元测试...
JUnit version 4.13.2
..E....E..
Time: 0.009
There were 2 failures:
1) testMultiBracket(TaxCalculatorTest)
java.lang.AssertionError: expected:<400.0> but was:<280.0>
        at org.junit.Assert.fail(Assert.java:89)
        at org.junit.Assert.failNotEquals(Assert.java:835)
        at org.junit.Assert.assertEquals(Assert.java:555)
        at org.junit.Assert.assertEquals(Assert.java:685)
        at TaxCalculatorTest.testMultiBracket(TaxCalculatorTest.java:37)
2) testFirstBracket(TaxCalculatorTest)
java.lang.AssertionError: expected:<10.0> but was:<0.0>
        at org.junit.Assert.fail(Assert.java:89)
        at org.junit.Assert.failNotEquals(Assert.java:835)
        at org.junit.Assert.assertEquals(Assert.java:555)
        at org.junit.Assert.assertEquals(Assert.java:685)
        at TaxCalculatorTest.testFirstBracket(TaxCalculatorTest.java:31)

FAILURES!!!
Tests run: 8,  Failures: 2
```

图 A.5 测试用例 TC5：收入为负数，抛出异常

```
请选择功能: 2
请输入新的起征点: 800
起征点已更新。

============= 个人所得税计算 =============
1. 计算个税
2. 设置起征点
3. 设置税率
0. 退出
============== 当前个税配置 ==============
起征点: 800.00
第 1 档:     0.00 -   500.00  税率:   5.00%
第 2 档:   500.00 -  2000.00  税率:  10.00%
第 3 档:  2000.00 -  5000.00  税率:  15.00%
第 4 档:  5000.00 - 20000.00  税率:  20.00%
第 5 档: 20000.00 -        !  税率:  25.00%
==========================================
```

图 A.6 调整起征点

```
请选择功能: 3
当前档位区间和税率如下:
第 1 档: 0 - 500.0, 税率 5.00%
第 2 档: 500 - 2000.0, 税率 10.00%
第 3 档: 2000 - 5000.0, 税率 15.00%
第 4 档: 5000 - 20000.0, 税率 20.00%
第 5 档: 20000 - 以上, 税率 25.00%
请选择设置方式:
1. 批量设置所有档位区间和税率
2. 单独设置某一档位的区间和税率
请输入选项（1或2, 其他返回菜单）: 2
当前共 5 档, 请输入要设置的档位编号（1-5）: 2
请输入该档位新的上限（inf 表示无穷）: 3000
请输入该档位新的税率（单位%）: 13
第2档区间和税率已更新。

============ 个人所得税计算 ============
1. 计算个税
2. 设置起征点
3. 设置税率
0. 退出
============ 当前个税配置 ==============
起征点: 800.00
第 1 档:     0.00 -   500.00  税率:   5.00%
第 2 档:   500.00 -  3000.00  税率:  13.00%
第 3 档:  3000.00 -  5000.00  税率:  15.00%
第 4 档:  5000.00 - 20000.00  税率:  20.00%
第 5 档: 20000.00 -        !  税率:  25.00%
========================================
```

图 A.7 调整各级税额计算

```
请选择功能: 3
当前档位区间和税率如下:
第 1 档: 0 - 500.0, 税率 5.00%
第 2 档: 500 - 3000.0, 税率 13.00%
第 3 档: 3000 - 5000.0, 税率 15.00%
第 4 档: 5000 - 20000.0, 税率 20.00%
第 5 档: 20000 - 以上, 税率 25.00%
请选择设置方式:
1. 批量设置所有档位区间和税率
2. 单独设置某一档位的区间和税率
请输入选项（1或2, 其他返回菜单）: 2
当前共 5 档, 请输入要设置的档位编号（1-5）: 2
请输入该档位新的上限（inf 表示无穷）: 6000
请输入该档位新的税率（单位%）: 14
Exception in thread "main" java.lang.IllegalArgumentException: 档位上限需递增且大于前一档
        at TaxConfig.setBrackets(TaxConfig.java:68)
        at CommandLineUI.handleSetRates(CommandLineUI.java:172)
        at CommandLineUI.start(CommandLineUI.java:41)
        at PersonalTaxApp.main(PersonalTaxApp.java:19)

 运行 JUnit 单元测试...
JUnit version 4.13.2
..E....E..
Time: 0.007
There were 2 failures:
1) testMultiBracket(TaxCalculatorTest)
java.lang.AssertionError: expected:<400.0> but was:<280.0>
        at org.junit.Assert.fail(Assert.java:89)
        at org.junit.Assert.failNotEquals(Assert.java:835)
        at org.junit.Assert.assertEquals(Assert.java:555)
        at org.junit.Assert.assertEquals(Assert.java:685)
        at TaxCalculatorTest.testMultiBracket(TaxCalculatorTest.java:37)
2) testFirstBracket(TaxCalculatorTest)
java.lang.AssertionError: expected:<10.0> but was:<0.0>
        at org.junit.Assert.fail(Assert.java:89)
        at org.junit.Assert.failNotEquals(Assert.java:835)
        at org.junit.Assert.assertEquals(Assert.java:555)
        at org.junit.Assert.assertEquals(Assert.java:685)
        at TaxCalculatorTest.testFirstBracket(TaxCalculatorTest.java:31)

FAILURES!!!
Tests run: 8,  Failures: 2
```

图 A.8 配置设置异常检测

A.2 代码

核心算法代码展示如下，采用 Java 语言实现：

A.2.1 TaxCalculator.java

Listing A.1 TaxCalculator.java

```java

/**
 * 个人所得税计算器类，支持超额累进税率计算。
 * <p>
 * 根据配置的税率表和起征点，计算应缴纳税额。
 * </p>
 * @author 林宏宇
 * @version 1.0
 */
public class TaxCalculator {

    /**
     * 税率和起征点配置。
     */
    private final TaxConfig config;

    /**
     * 构造计算器对象。
     * @param config 税率配置
     */
    public TaxCalculator(TaxConfig config) {
        this.config = config;
    }

    /**
     * 计算指定收入的应纳税额。
     * @param income 月收入
     * @return 应缴税额
     * @throws IllegalArgumentException 收入为负时抛出
     */
    public double computeTax(double income) {
```

```
32          if (income < 0) {
33              throw new IllegalArgumentException("Income must be non-negative."
    );
34          }
35          double taxable = income - config.getThreshold();
36          if (taxable <= 0) {
37              return 0.0;
38          }
39
40          double tax = 0.0;
41          double previousUpper = 0.0;
42          for (TaxBracket bracket : config.getBrackets()) {
43              double upper = bracket.getUpperBound();
44              double portion = Math.min(taxable, upper) - previousUpper;
45              if (portion > 0) {
46                  tax += portion * bracket.getRate();
47              }
48              if (taxable <= upper) {
49                  break;
50              }
51              previousUpper = upper;
52          }
53          return tax;
54      }
55  }
```

A.2.2 TaxConfig.java

Listing A.2 TaxConfig.java

```
1
2   /**
3    * 税率配置类，包含起征点和所有税率档次。
4    * <p>
5    * 支持动态调整起征点和税率。
6    * </p>
7    * @author 林宏宇
8    * @version 1.0
```

```
 */
public class TaxConfig {
    /**
     * 个人所得税起征点。
     */
    private double threshold;

    /**
     * 所有税率档次。
     */
    private TaxBracket[] brackets;

    /**
     * 构造默认税率配置。
     */
    public TaxConfig() {
        this.threshold = 1600.0;
        this.brackets = defaultBrackets();
    }

    /**
     * 获取当前起征点。
     * @return 起征点
     */
    public double getThreshold() {
        return threshold;
    }

    /**
     * 设置起征点。
     * @param threshold 新的起征点
     */
    public void setThreshold(double threshold) {
        if (threshold < 0) {
            throw new IllegalArgumentException("Threshold must be non-
    negative.");
        }
        this.threshold = threshold;
```

```
46      }
47
48      /**
49       * 获取所有税率档次。
50       * @return 税率档数组
51       */
52      public TaxBracket[] getBrackets() {
53          return brackets;
54      }
55
56
57      /**
58       * 设置所有税率和区间。
59       * @param uppers 长度为5的上限数组，最后一档应为Double.POSITIVE_INFINITY
60       * @param rates 长度为5的税率数组
61       */
62      public void setBrackets(double[] uppers, double[] rates) {
63          if (uppers == null || rates == null || uppers.length != 5 || rates.
    length != 5) {
64              throw new IllegalArgumentException("档位区间和税率数组长度必须为5
    。");
65          }
66          double last = 0;
67          for (int i = 0; i < 4; i++) {
68              if (uppers[i] <= last) throw new IllegalArgumentException("档位上
    限需递增且大于前一档");
69              last = uppers[i];
70          }
71          if (uppers[4] != Double.POSITIVE_INFINITY) throw new
    IllegalArgumentException("最后一档上限必须为无穷大");
72          this.brackets = new TaxBracket[] {
73              new TaxBracket(uppers[0], rates[0]),
74              new TaxBracket(uppers[1], rates[1]),
75              new TaxBracket(uppers[2], rates[2]),
76              new TaxBracket(uppers[3], rates[3]),
77              new TaxBracket(uppers[4], rates[4])
78          };
79      }
```

```
80
81      /**
82      * 兼容原有，仅设置税率，区间不变。
83      * @param rates 新的税率数组（长度需为5，单位为小数，如0.05表示5%）
84       */
85      public void setRates(double[] rates) {
86          if (rates == null || rates.length != 5) {
87              throw new IllegalArgumentException("Rates length must be 5.");
88          }
89          TaxBracket[] old = this.brackets;
90          this.brackets = new TaxBracket[] {
91              new TaxBracket(old[0].getUpperBound(), rates[0]),
92              new TaxBracket(old[1].getUpperBound(), rates[1]),
93              new TaxBracket(old[2].getUpperBound(), rates[2]),
94              new TaxBracket(old[3].getUpperBound(), rates[3]),
95              new TaxBracket(old[4].getUpperBound(), rates[4])
96          };
97      }
98
99      /**
100      * 获取默认税率档数组。
101      * @return 默认税率档
102      */
103     public static TaxBracket[] defaultBrackets() {
104         return new TaxBracket[] {
105             new TaxBracket(500.0, 0.05),
106             new TaxBracket(2000.0, 0.10),
107             new TaxBracket(5000.0, 0.15),
108             new TaxBracket(20000.0, 0.20),
109             new TaxBracket(Double.POSITIVE_INFINITY, 0.25)
110         };
111     }
112 }
```

A.2.3 TaxBracket.java

Listing A.3 TaxBracket.java

```java

/**
 * 税率档次类，表示超额累进税率的一个区间。
 * <p>
 * 每个对象包含该档的上限和税率。
 * </p>
 * @author 林宏宇
 * @version 1.0
 */
public class TaxBracket {
    /**
     * 本档税率的上限（含）。如为Double.POSITIVE_INFINITY表示无上限。
     */
    private final double upperBound;

    /**
     * 本档适用税率（如0.05表示5%）。
     */
    private final double rate;

    /**
     * 构造一个税率档对象。
     * @param upperBound 本档上限
     * @param rate 本档税率
     */
    public TaxBracket(double upperBound, double rate) {
        this.upperBound = upperBound;
        this.rate = rate;
    }

    /**
     * 获取本档上限。
     * @return 上限数值
     */
    public double getUpperBound() {
        return upperBound;
    }
```

```
38
39      /**
40       * 获取本档税率。
41       * @return 税率
42       */
43      public double getRate() {
44          return rate;
45      }
46  }
```

A.2.4 CommandLineUI.java

Listing A.4 CommandLineUI.java

```
1   /**
2    * 命令行用户交互界面类。
3    * <p>
4    * 负责菜单显示、用户输入、功能分发和异常处理。
5    * </p>
6    * @author 林宏宇
7    * @version 1.0
8    */
9   public class CommandLineUI {
10      private final java.util.Scanner scanner;
11      private final TaxConfig config;
12      private final TaxCalculator calculator;
13
14      /**
15       * 构造命令行界面对象。
16       * @param config 税率配置
17       */
18      public CommandLineUI(TaxConfig config) {
19          this.config = config;
20          this.calculator = new TaxCalculator(config);
21          this.scanner = new java.util.Scanner(System.in);
22      }
23
24      /**
```

```
25      * 启动主菜单循环。
26      */
27     public void start() {
28         boolean running = true;
29         while (running) {
30             printMenu();
31             printConfig(); // 每次菜单后自动展示配置信息
32             int choice = readInt("请选择功能：");
33             switch (choice) {
34                 case 1:
35                     handleCompute();
36                     break;
37                 case 2:
38                     handleSetThreshold();
39                     break;
40                 case 3:
41                     handleSetRates();
42                     break;
43                 case 0:
44                     running = false;
45                     System.out.println("已退出。");
46                     break;
47                 default:
48                     System.out.println("无效选择，请重试。");
49             }
50         }
51     }
52 
53     private void printMenu() {
54         System.out.println("\n============== 个人所得税计算 ==============");
55         System.out.println("1. 计算个税");
56         System.out.println("2. 设置起征点");
57         System.out.println("3. 设置税率");
58         System.out.println("0. 退出");
59     }
60 
61     private void handleCompute() {
62         double income = readDouble("请输入当月工资总额：");
```

```
63          double tax = calculator.computeTax(income);
64          System.out.printf("应缴纳个人所得税：%.2f 元\n", tax);
65      }
66
67      private void handleSetThreshold() {
68          double threshold = readDouble("请输入新的起征点：");
69          try {
70              config.setThreshold(threshold);
71              System.out.println("起征点已更新。");
72          } catch (IllegalArgumentException ex) {
73              System.out.println("输入无效：" + ex.getMessage());
74          }
75      }
76
77      private void handleSetRates() {
78          System.out.println("当前档位区间和税率如下：");
79          TaxBracket[] brackets = config.getBrackets();
80          double lower = 0.0;
81          for (int i = 0; i < brackets.length; i++) {
82              TaxBracket b = brackets[i];
83              String upperText = b.getUpperBound() == Double.POSITIVE_INFINITY
    ? "以上" : String.valueOf(b.getUpperBound());
84              System.out.printf("第 %d 档：%.0f - %s，税率 %.2f%%\n", i + 1,
    lower, upperText, b.getRate() * 100);
85              lower = b.getUpperBound();
86          }
87          System.out.println("请选择设置方式：");
88          System.out.println("1. 批量设置所有档位区间和税率");
89          System.out.println("2. 单独设置某一档位的区间和税率");
90          System.out.print("请输入选项（1或2，其他返回菜单）：");
91          String opt = scanner.nextLine().trim();
92          if (opt.equals("1")) {
93              System.out.println("请输入各档位的上限（以逗号分隔，最后一档填 inf
    ）：");
94              String upperLine = scanner.nextLine().trim();
95              String[] upperStrs = upperLine.split(",");
96              double[] uppers = new double[upperStrs.length];
97              for (int i = 0; i < upperStrs.length; i++) {
```

```
98                  if (upperStrs[i].trim().equalsIgnoreCase("inf")) {
99                      uppers[i] = Double.POSITIVE_INFINITY;
100                 } else {
101                     try {
102                         uppers[i] = Double.parseDouble(upperStrs[i].trim());
103                     } catch (NumberFormatException e) {
104                         System.out.println("上限输入有误，请重新设置。");
105                         return;
106                     }
107                 }
108             }
109             System.out.println("请输入各档位的税率（以逗号分隔，单位%）：");
110             String rateLine = scanner.nextLine().trim();
111             String[] rateStrs = rateLine.split(",");
112             if (rateStrs.length != uppers.length) {
113                 System.out.println("税率数量与档位数量不符，请重新设置。");
114                 return;
115             }
116             double[] rates = new double[rateStrs.length];
117             for (int i = 0; i < rateStrs.length; i++) {
118                 try {
119                     rates[i] = Double.parseDouble(rateStrs[i].trim()) /
    100.0;
120                 } catch (NumberFormatException e) {
121                     System.out.println("税率输入有误，请重新设置。");
122                     return;
123                 }
124             }
125             config.setBrackets(uppers, rates);
126             System.out.println("档位区间和税率已批量更新。");
127         } else if (opt.equals("2")) {
128             System.out.printf("当前共 %d 档，请输入要设置的档位编号（1-%d）："
    , brackets.length, brackets.length);
129             String idxStr = scanner.nextLine().trim();
130             int idx;
131             try {
132                 idx = Integer.parseInt(idxStr) - 1;
133             } catch (NumberFormatException e) {
```

```
134                 System.out.println("编号输入有误。");
135                 return;
136             }
137             if (idx < 0 || idx >= brackets.length) {
138                 System.out.println("编号超出范围。");
139                 return;
140             }
141             System.out.print("请输入该档位新的上限（inf 表示无穷）：");
142             String upperStr = scanner.nextLine().trim();
143             double upper;
144             if (upperStr.equalsIgnoreCase("inf")) {
145                 upper = Double.POSITIVE_INFINITY;
146             } else {
147                 try {
148                     upper = Double.parseDouble(upperStr);
149                 } catch (NumberFormatException e) {
150                     System.out.println("上限输入有误。");
151                     return;
152                 }
153             }
154             System.out.print("请输入该档位新的税率（单位%）：");
155             String rateStr = scanner.nextLine().trim();
156             double rate;
157             try {
158                 rate = Double.parseDouble(rateStr) / 100.0;
159             } catch (NumberFormatException e) {
160                 System.out.println("税率输入有误。");
161                 return;
162             }
163             // 构造新的档位数组
164             double[] uppers = new double[brackets.length];
165             double[] rates = new double[brackets.length];
166             for (int i = 0; i < brackets.length; i++) {
167                 uppers[i] = brackets[i].getUpperBound();
168                 rates[i] = brackets[i].getRate();
169             }
170             uppers[idx] = upper;
171             rates[idx] = rate;
```

```
172             config.setBrackets(uppers, rates);
173             System.out.println("第" + (idx + 1) + "档区间和税率已更新。");
174         } else {
175             System.out.println("返回主菜单。");
176         }
177     }
178 
179     private void printConfig() {
180         System.out.println("============== 当前个税配置 ==============");
181         System.out.printf("起征点：%.2f\n", config.getThreshold());
182         TaxBracket[] brackets = config.getBrackets();
183         double lower = 0.0;
184         for (int i = 0; i < brackets.length; i++) {
185             TaxBracket b = brackets[i];
186             String upperText = b.getUpperBound() == Double.POSITIVE_INFINITY
    ? "!" : String.format("%.2f", b.getUpperBound());
187             System.out.printf("第 %d 档：%8s - %8s  税率：%6.2f%%\n",
188                 i + 1, String.format("%.2f", lower), upperText, b.getRate() *
     100);
189             lower = b.getUpperBound();
190         }
191         System.out.println("=========================================");
192     }
193 
194     private int readInt(String prompt) {
195         while (true) {
196             System.out.print(prompt);
197             String line = scanner.nextLine().trim();
198             try {
199                 return Integer.parseInt(line);
200             } catch (NumberFormatException ex) {
201                 System.out.println("请输入整数。");
202             }
203         }
204     }
205 
206     private double readDouble(String prompt) {
207         while (true) {
```

```
208             System.out.print(prompt);
209             String line = scanner.nextLine().trim();
210             try {
211                 return Double.parseDouble(line);
212             } catch (NumberFormatException ex) {
213                 System.out.println("请输入数字。");
214             }
215         }
216     }
217 }
```

A.2.5 PersonalTaxApp.java

Listing A.5 PersonalTaxApp.java

```
1
2  /**
3   * 个人所得税计算程序主入口类。
4   * <p>
5   * 启动配置和命令行界面。
6   * </p>
7   * @author 林宏宇
8   * @version 1.0
9   */
10 public class PersonalTaxApp {
11
12     /**
13      * 程序主方法，启动个税计算系统。
14      * @param args 命令行参数
15      */
16     public static void main(String[] args) {
17         TaxConfig config = new TaxConfig();
18         CommandLineUI ui = new CommandLineUI(config);
19         ui.start();
20     }
21 }
```